CS413 Midterm 1

# Compiling Extended LAMBDA into Pure Lambda Calculus

Church encodings, call-by-value reduction, and bounded observation

Dino Redzepagic · Fall 2026

# What this project does

The goal is to take a normal-looking LAMBDA program and turn it into pure lambda calculus.

That sounds restrictive because the final program can only use variables, one-argument functions, and function calls.

My compiler keeps the program's meaning while removing built-in numbers, booleans, conditionals, pairs, and recursion.

# The two languages

The source language feels familiar: it has numbers, true and false, math, if statements, let bindings, pairs, and recursive functions.

The target language only has Var, Lam, and App. In other words: a variable, a function, or a function call.

The main question is: can those three pieces represent everything in the source program? This project shows that they can.

# How the pieces fit together

First, the source AST goes into my visitor-based compiler.

Then the compiler produces a pure lambda term, and the reducer runs that term with a step limit.

Finally, an observation context turns the Church-encoded result back into a number, boolean, or pair that we can check.

# Church encodings

For booleans, true chooses the first answer and false chooses the second: TRUE = λt. λf. t and FALSE = λt. λf. f.

A Church number is a function that repeats an action. For example, 2 means apply the action twice.

Pairs, arithmetic, and comparisons are also built from functions. Nothing in the target language treats them as special.

The predecessor function carries a little bit of state so that predecessor of zero stays zero.

# Why call-by-value matters

Call-by-value means the program evaluates a function's argument before it calls the function.

A let becomes (λx. body) value, which gives the same evaluation order as the source language.

For if, I put each branch inside a small function called a thunk. The boolean chooses one thunk, and only that thunk runs.

That is why if true then 1 else LOOP returns 1 instead of getting stuck in LOOP. Recursion uses the call-by-value Z combinator.

# Variables and safe substitution

When a lambda function receives an argument, beta reduction substitutes the argument into the body.

The tricky part is avoiding accidental variable capture. If a name would collide, the reducer gives the binder a fresh name first.

The compiler uses fresh names such as thunk_1, but skips any names already used by the source program.

That keeps scope correct for shadowing, open expressions, and even the unusual case where both fix binders have the same name.

# Running and decoding programs

The reducer works left to right and does not run code inside a lambda body. A lambda is already a finished value.

Every beta reduction uses one step from a budget. When the budget runs out, I report exhaustion rather than claiming the program is definitely infinite.

To read a number, I run it with a special step and base function, then count the resulting structure.

For booleans, I give it two different marker functions. For pairs, I project and decode each side.

# Testing the implementation

The test suite has 10 test groups. It checks substitution, zero-step values, step limits, every required operator, pairs, recursion, invalid syntax, and free variables.

It also checks evaluation order. False and LOOP must stop immediately, but a LOOP passed as an ordinary argument still uses up the budget.

The demo runs all required examples. Some results are 2 + 3 × 4 = 14, factorial 4 = 24, and Fibonacci 5 = 5.

# What worked and what I left out

The required natural-number and boolean language compiles and runs through pure lambda terms.

Terminating demo programs use a budget of 1,000,000 steps. Intentional LOOP examples use a smaller budget of 200 steps.

I did not add a parser, type checker, signed integers, division, lists, or reduction under lambda bodies because the assignment does not require them.

The tests support the design, but a formal correctness proof would need a separate mathematical argument.

# Takeaway

This project shows that a tiny language based only on functions can represent much more familiar programming features.

Church encodings provide the data. Thunks preserve conditional and short-circuit behavior. Z makes recursive functions possible under call-by-value.

Questions